Experiment No. 5
Hands-on Solidity Programming (Remix IDE)

Aim

To perform hands-on Solidity programming assignments for creating smart contracts, and to compile and deploy them on the Remix VM using the LearnEth plugin.

Theory

Solidity is a statically typed, contract-oriented language used to write smart contracts for the Ethereum Virtual Machine (EVM). Remix IDE is a browser-based IDE to write, compile, deploy and test contracts; the Remix VM is an in-browser blockchain used for testing without real ether.

Key concepts covered in this experiment:

- **Data types and variables** – `uint`, `int`, `bool`, `address`, `string`; state, local and global variables.
- **Functions** – `view` (reads state) and `pure` (no state access); modifiers and constructors.
- **Visibility** – `public`, `private`, `internal`, `external`.
- **Control flow** – if/else, ternary operator, loops with `break` and `continue`.
- **Data structures** – arrays, mappings, structs, enums.
- **Data locations** – `storage`, `memory`, `calldata`.
- **Transactions** – ether and wei units, gas and gas price, sending ether using `payable` and `call`.

Units of ether:

Unit	Value in wei	Solidity keyword
wei	1	`1 wei`
gwei	10^9	`1 gwei`
ether	10^{18}	`1 ether`

Requirements

- Computer with a web browser and internet connection.
- Remix IDE (`https://remix.ethereum.org`) – Remix VM (Osaka) as environment.
- Solidity compiler 0.8.x.
- LearnEth plugin – Solidity Beginner Course.

Procedure

Step 1: Open Remix IDE

Visit `https://remix.ethereum.org` in the browser.

Step 2: Activate LearnEth

Open the Plugin Manager, activate LearnEth and select the Solidity Beginner Course.

Step 3: Solve the Tutorials

Open each tutorial in order, write the required code in the editor, compile it from the Solidity Compiler tab and deploy it on the Remix VM where required.

Step 4: Check the Answer

Click *Check Answer*. Every tutorial returned *Well done! No errors.*

Programs Executed

The code of each tutorial and the Remix screenshot are given below.

Tutorial 1: Compile, Deploy and Interact (Counter)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract Counter {
    uint public count;
    function get() public view returns (uint) { return count; }
    function inc() public { count += 1; }
    function dec() public { count -= 1; }
}
```

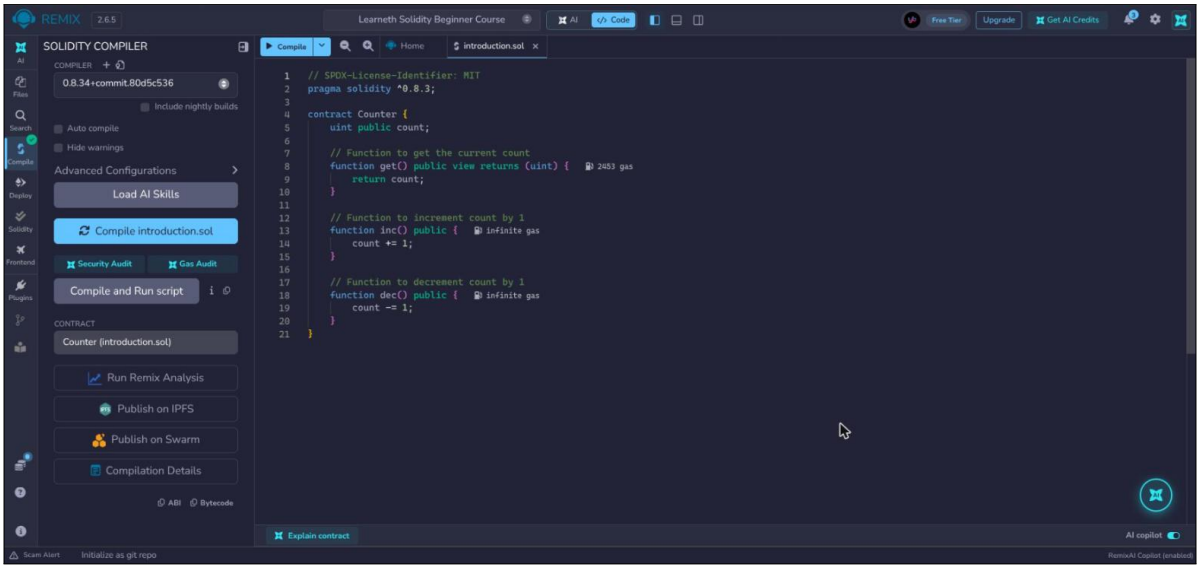


Figure 1: Compiling the Counter contract

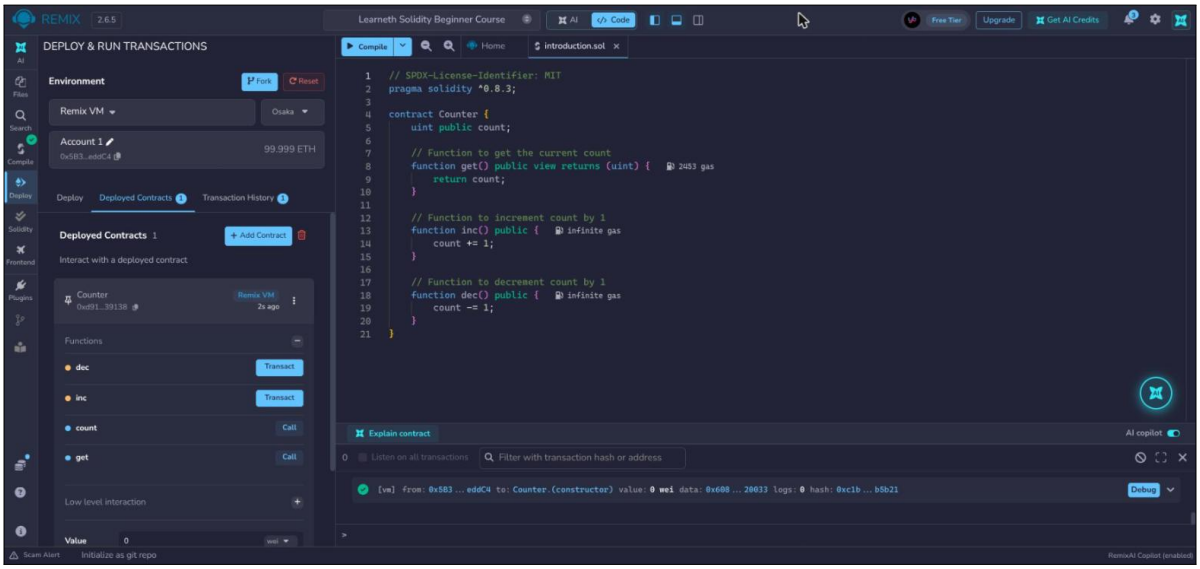


Figure 2: Deploying Counter on Remix VM

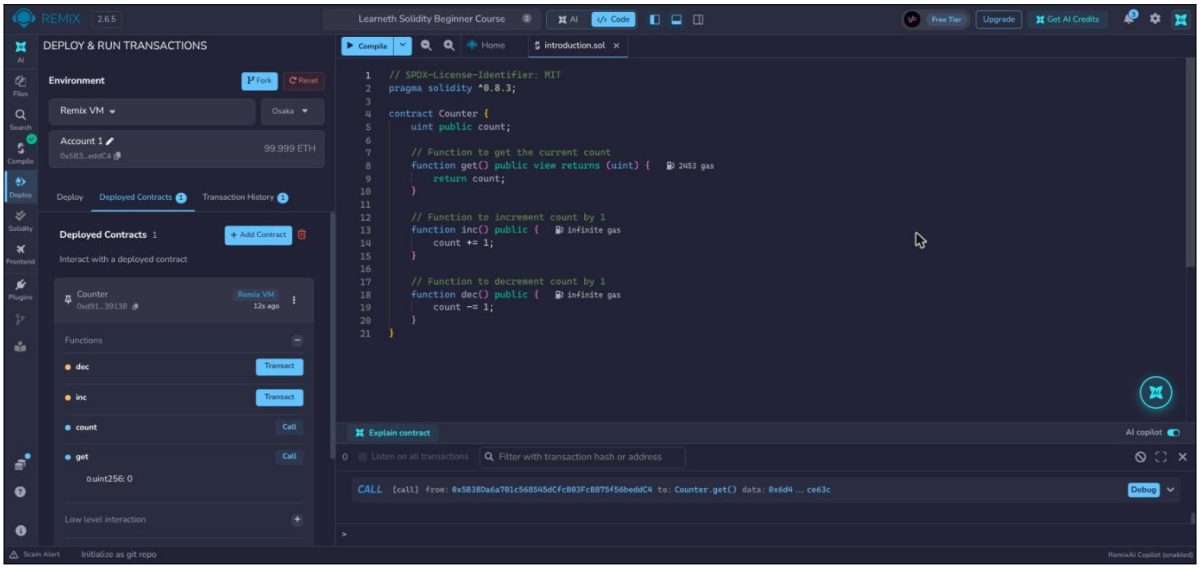


Figure 3: `get()` returns 0

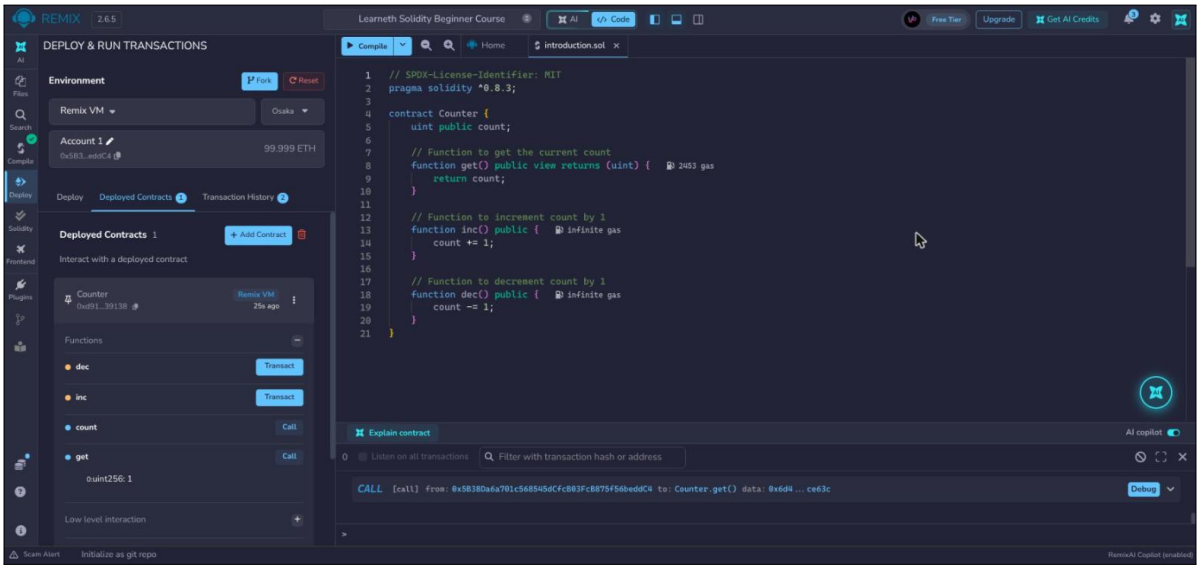


Figure 4: `inc()` called, `get()` returns 1

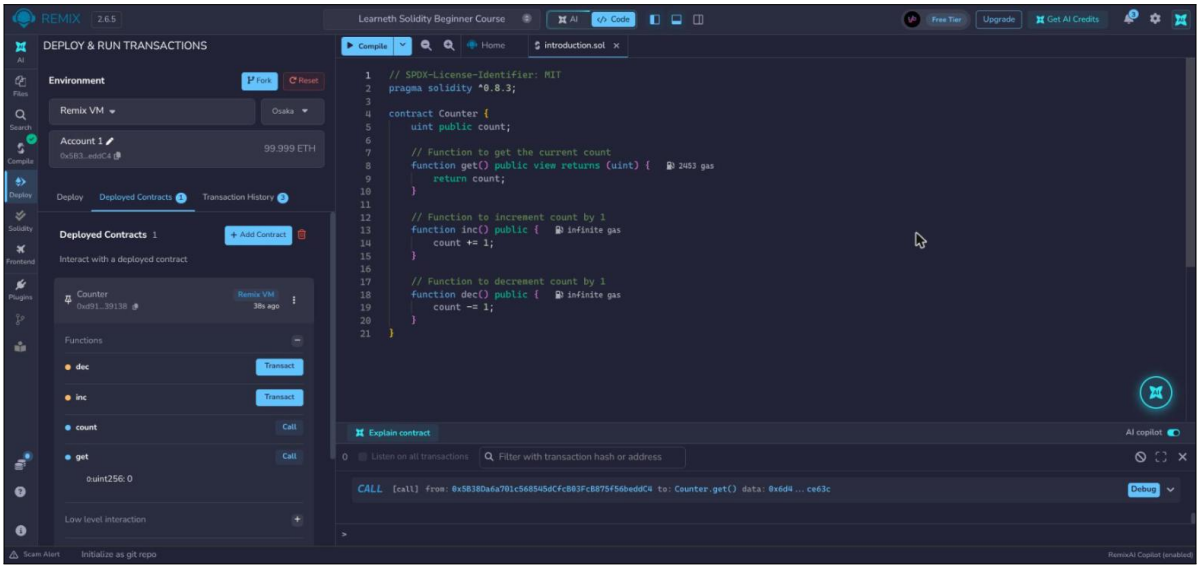


Figure 5: `dec()` called, `get()` returns 0

Tutorial 2: Basic Syntax

```
contract MyContract {
    string public name = "Alice";
}
```

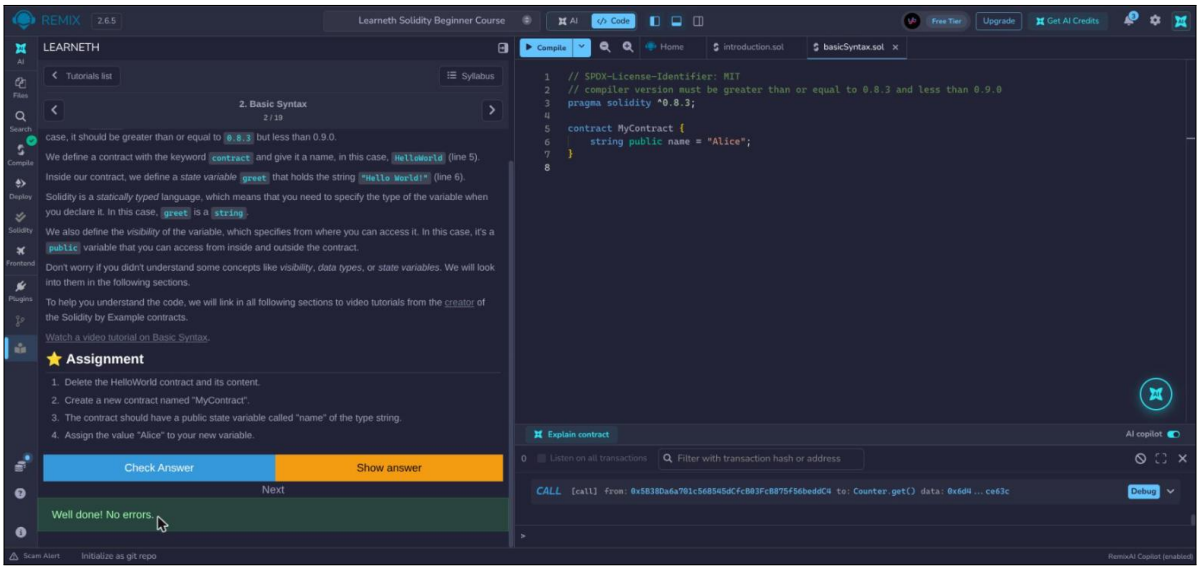


Figure 6: Basic syntax – MyContract

Tutorial 3: Primitive Data Types

```
address public newAddr = 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4;
int public neg = -12;
uint8 public newU = 0;
```

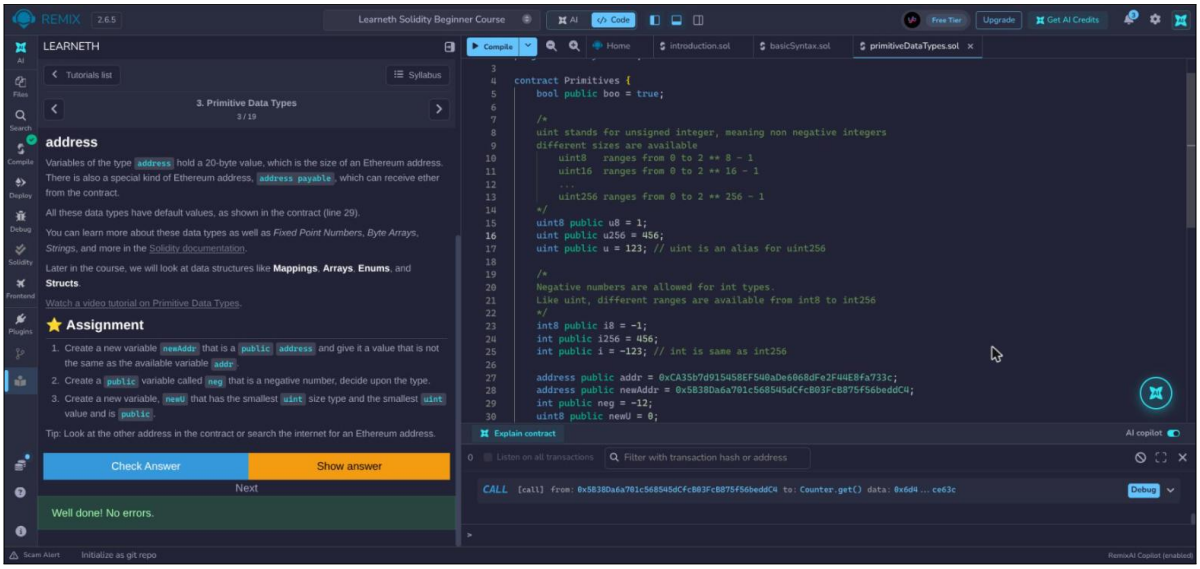


Figure 7: Primitive data types

Tutorial 4: Variables

```
uint public blockNumber;
function doSomething() public {
    blockNumber = block.number; // global variable
}
```

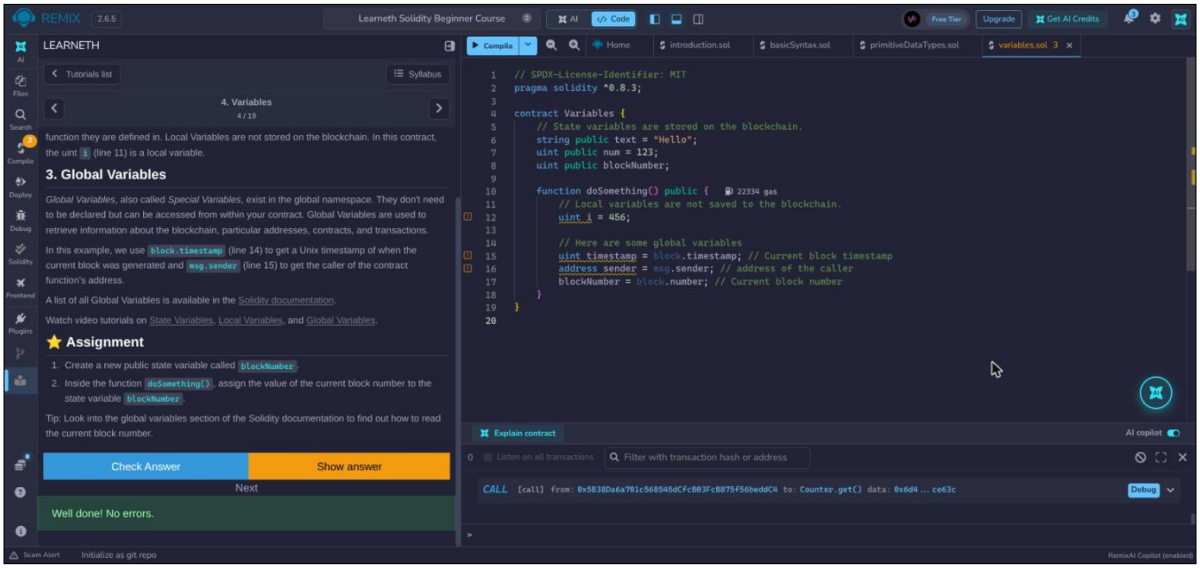


Figure 8: State, local and global variables

Tutorial 5: Functions

5.1 Reading and Writing

```
bool public b = true;
function get_b() public view returns (bool) { return b; }
```

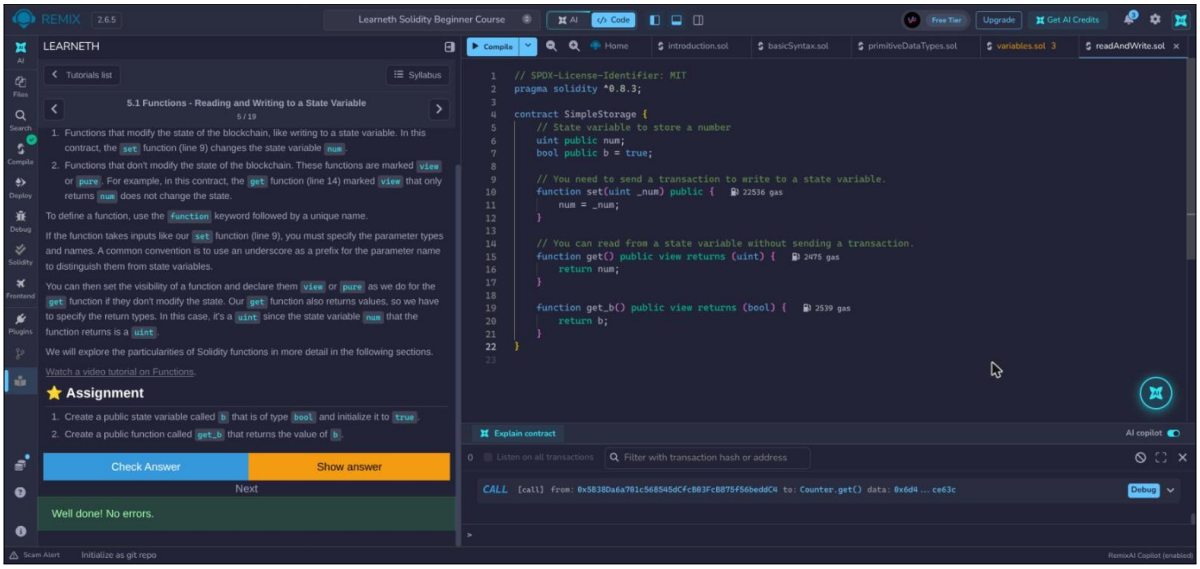


Figure 9: Functions – reading and writing to state

5.2 View and Pure

```
function addToX2(uint y) public { x = x + y; }
```

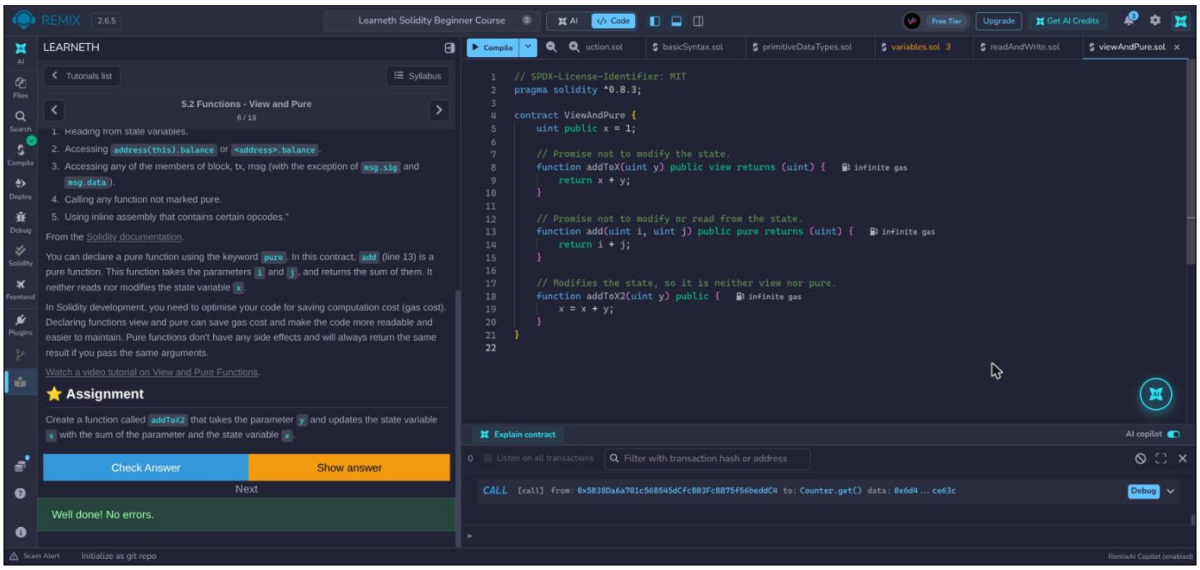


Figure 10: View and pure functions

5.3 Modifiers and Constructors

```
modifier increase(uint _x) {
    require(_x > 0, "x can only be increased");
    x += _x;
    _;
}
function increaseX(uint _x) public increase(_x) {}
```

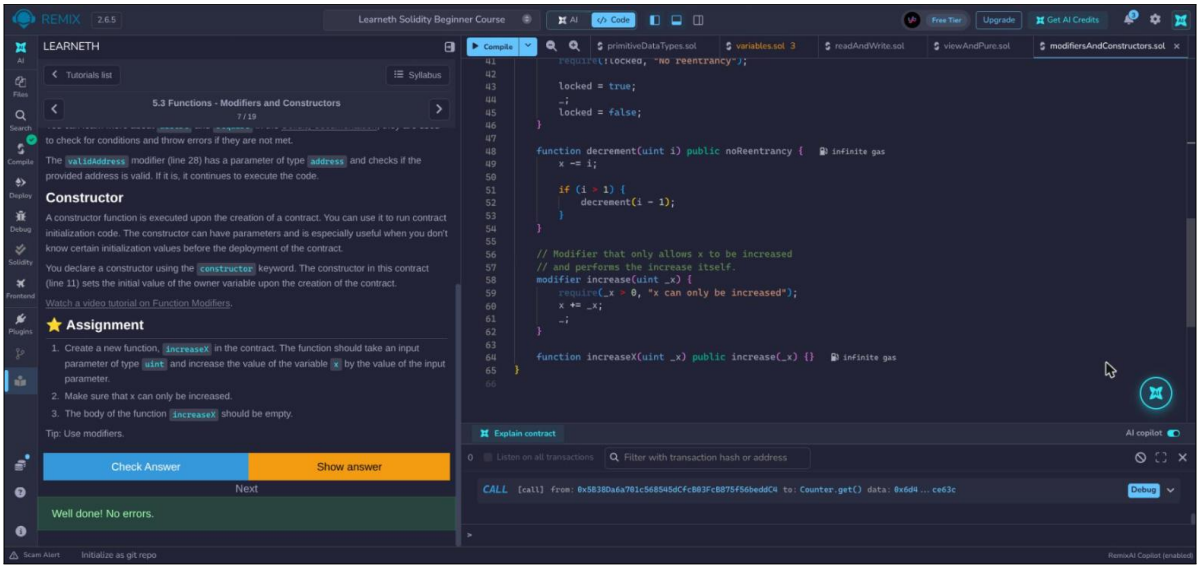


Figure 11: Modifiers and constructors

5.4 Inputs and Outputs

```
function returnTwo() public pure returns (int x, bool b) {
    x = -2;
    b = true;
}
```

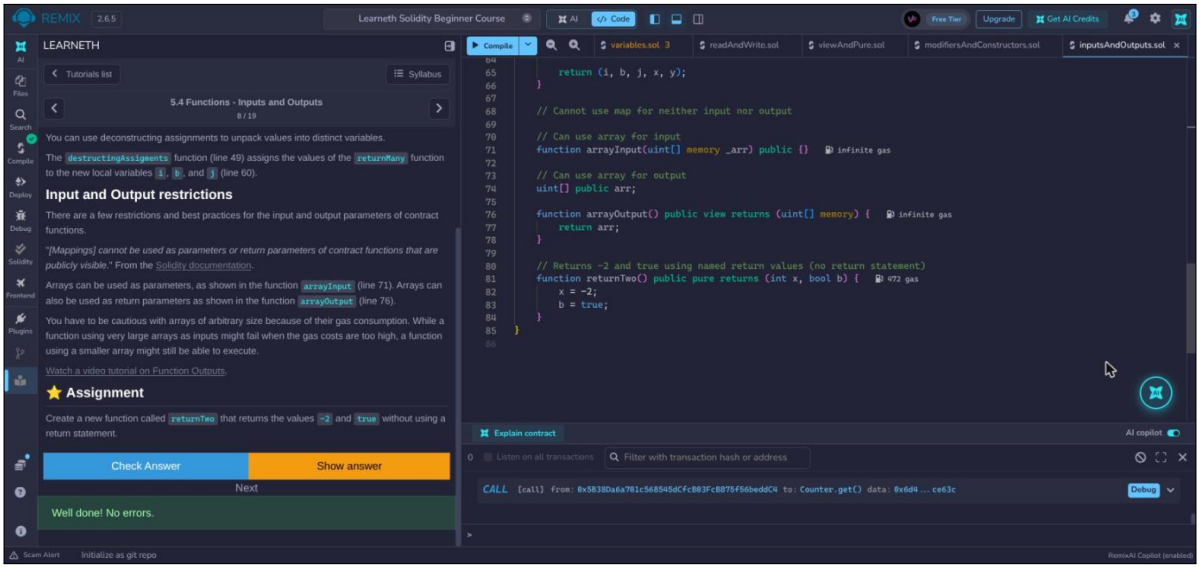


Figure 12: Function inputs and outputs

Tutorial 6: Visibility

```
function testInternalVar() public view returns (string memory, string
    memory) {
     return (internalVar, publicVar);
}
```

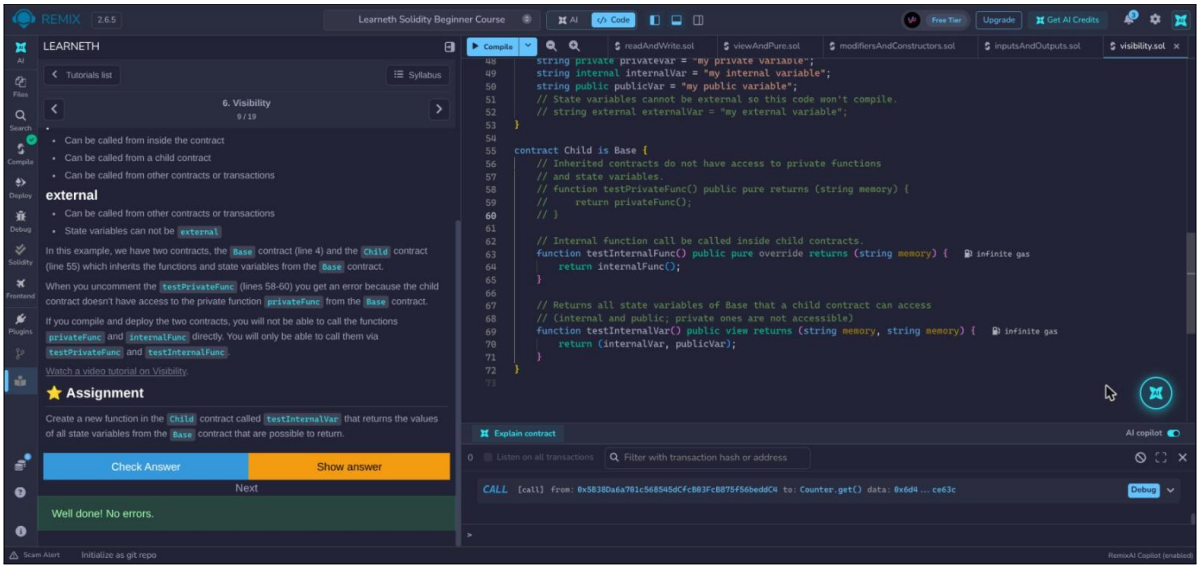


Figure 13: Visibility – Child contract reading inherited variables

Tutorial 7: Control Flow

7.1 If/Else

```
function evenCheck(uint y) public pure returns (bool) {
    return y % 2 == 0 ? true : false;
}
```

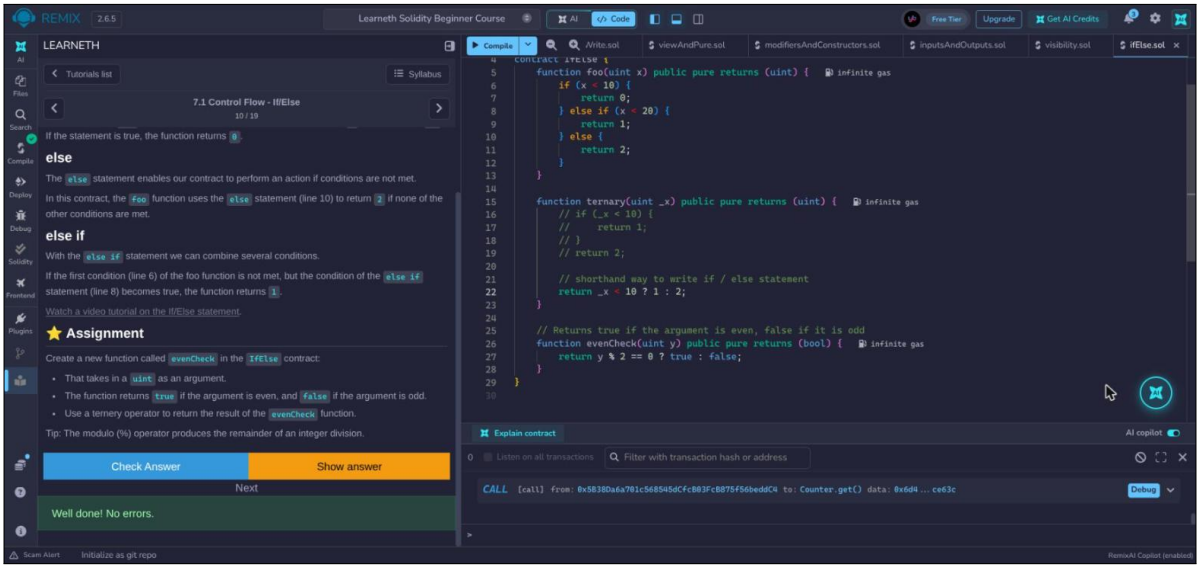


Figure 14: If/else using the ternary operator

7.2 Loops

```
uint public count;
for (uint i = 6; i < 15; i++) {
    if (i == 3) { continue; }
    if (i == 5) { break; }
    count++;
}
```

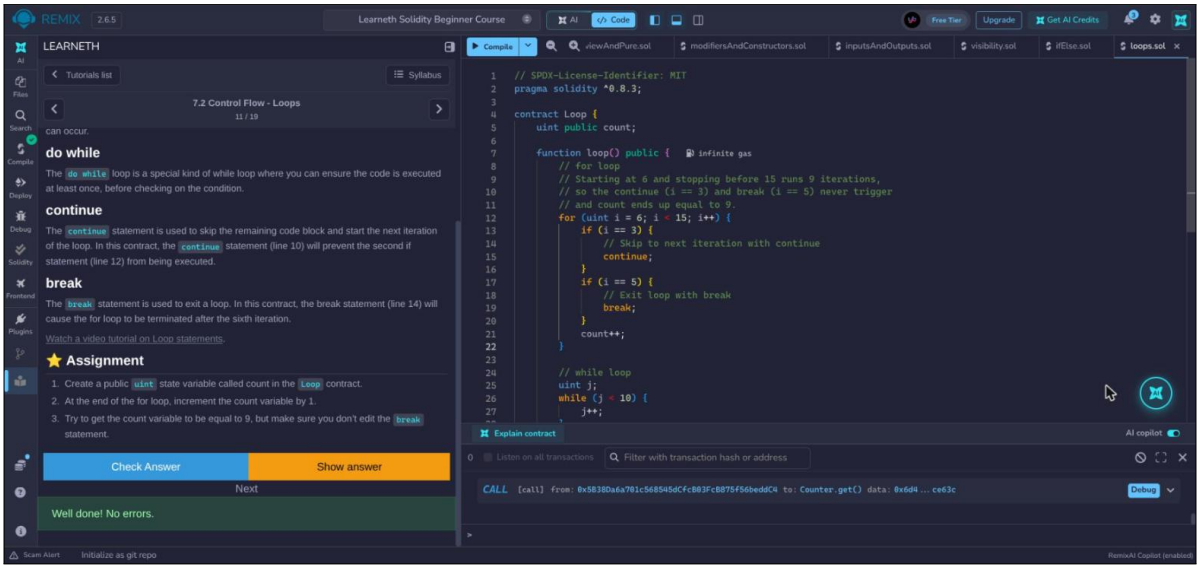


Figure 15: Loops with continue and break

Tutorial 8: Data Structures

8.1 Arrays

```
uint[3] public arr3 = [0, 1, 2];
function getArr() public view returns (uint[3] memory) { return arr3; }
```

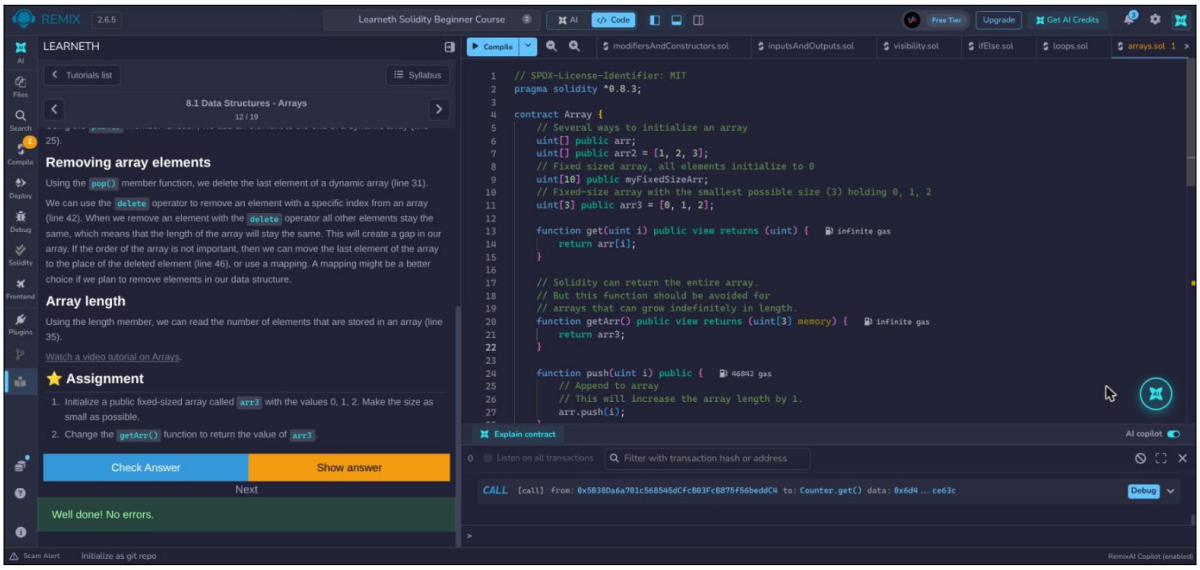


Figure 16: Arrays

8.2 Mappings

```
mapping(address => uint) public balances;
function get(address _addr) public view returns (uint) { return
    balances[_addr]; }
function set(address _addr) public { balances[_addr] = _addr.balance; }
function remove(address _addr) public { delete balances[_addr]; }
```

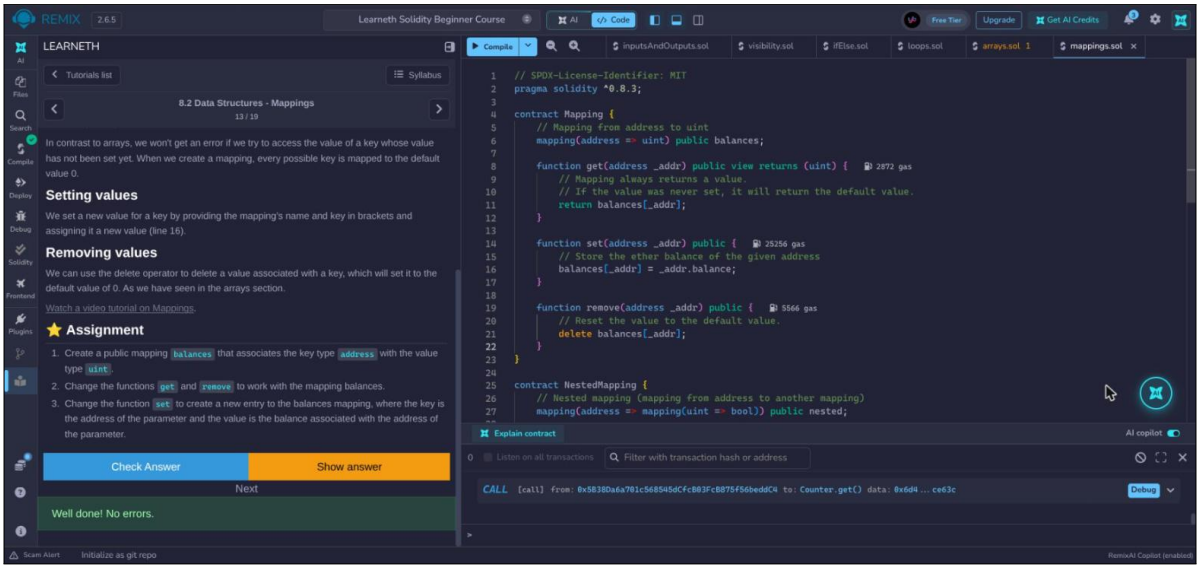


Figure 17: Mappings

8.3 Structs

```
function remove(uint _index) public { delete todos[_index]; }
```

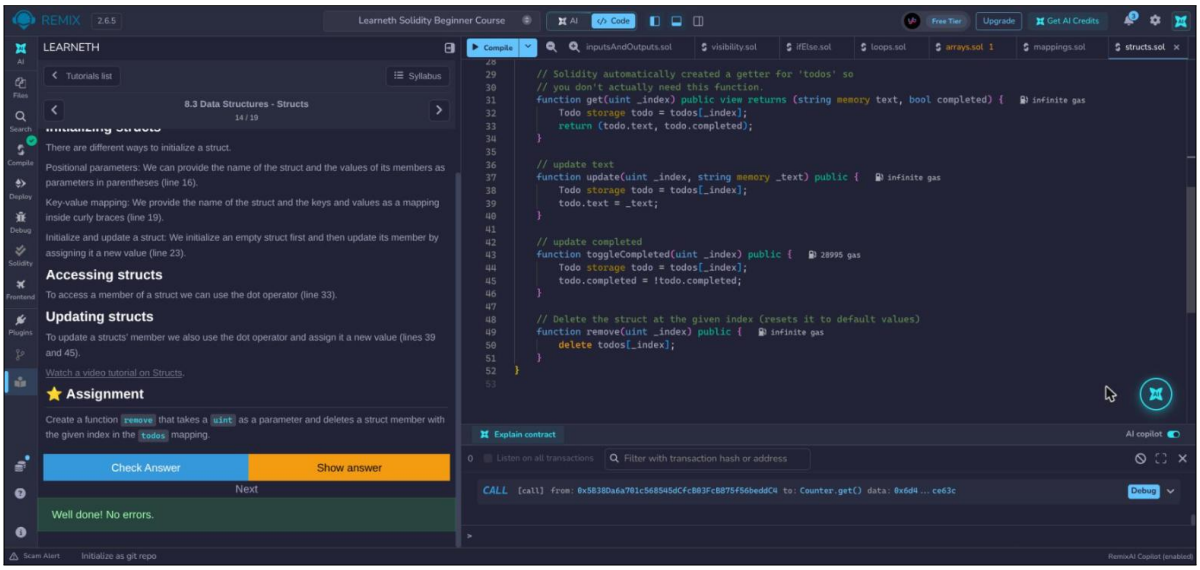


Figure 18: Structs

8.4 Enums

```
enum Size { S, M, L }
Size public sizes;
function getSize() public view returns (Size) { return sizes; }
```

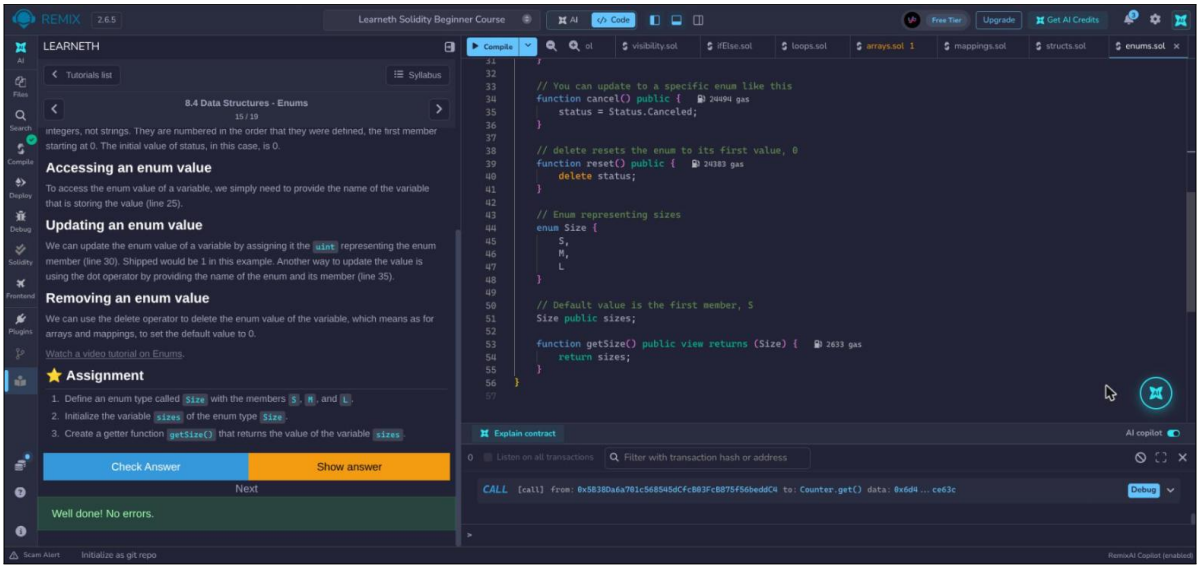


Figure 19: Enums

Tutorial 9: Data Locations

```
MyStruct storage myStruct = myStructs[1];
myStruct.foo = 4;
MyStruct memory myMemStruct = MyStruct(0);
MyStruct memory myMemStruct2 = myMemStruct;
myMemStruct2.foo = 1;
MyStruct memory myMemStruct3 = myStruct;
myMemStruct3.foo = 3;
return (myStruct, myMemStruct2, myMemStruct3);
```

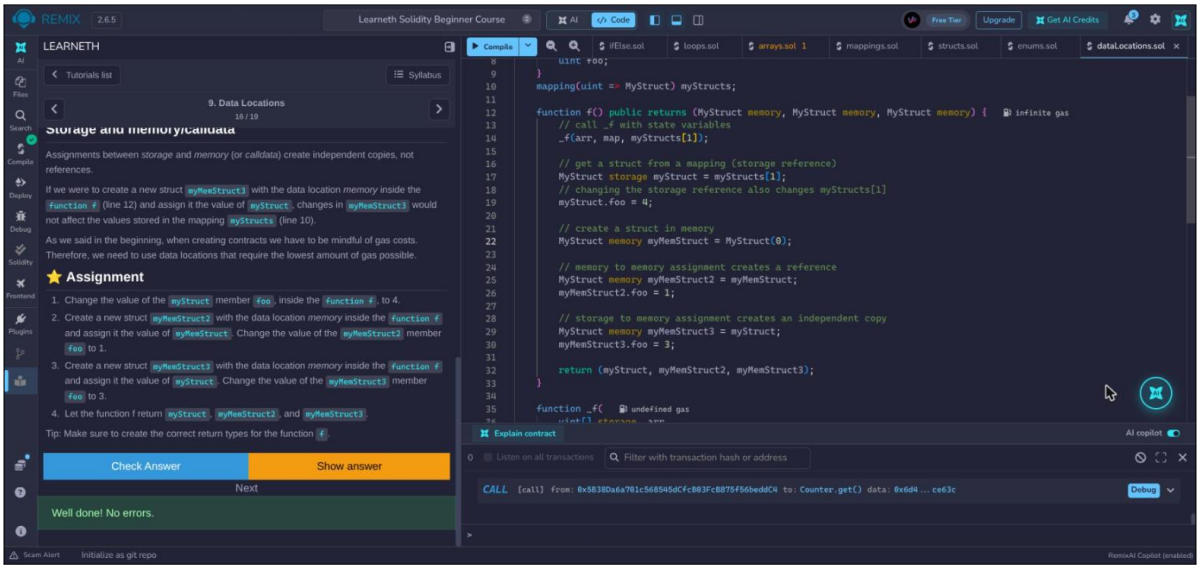


Figure 20: Storage vs memory

Tutorial 10: Transactions

10.1 Ether and Wei

```
uint public oneGwei = 1 gwei;
bool public isOneGwei = 1 gwei == 1e9;
```

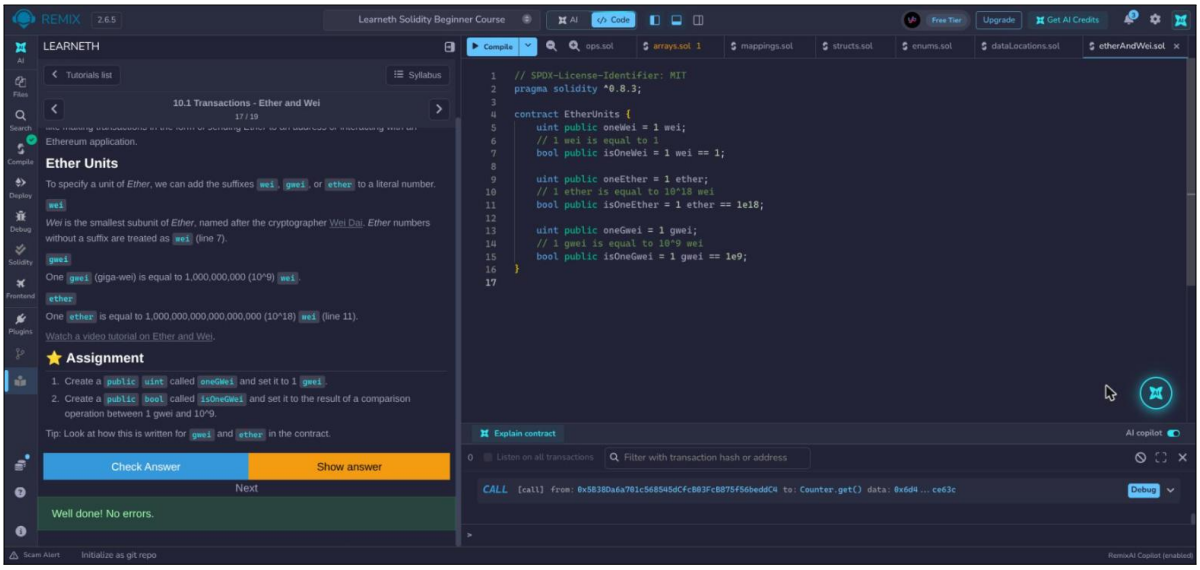


Figure 21: Ether and wei

10.2 Gas and Gas Price

```
uint public cost = 170367;
```

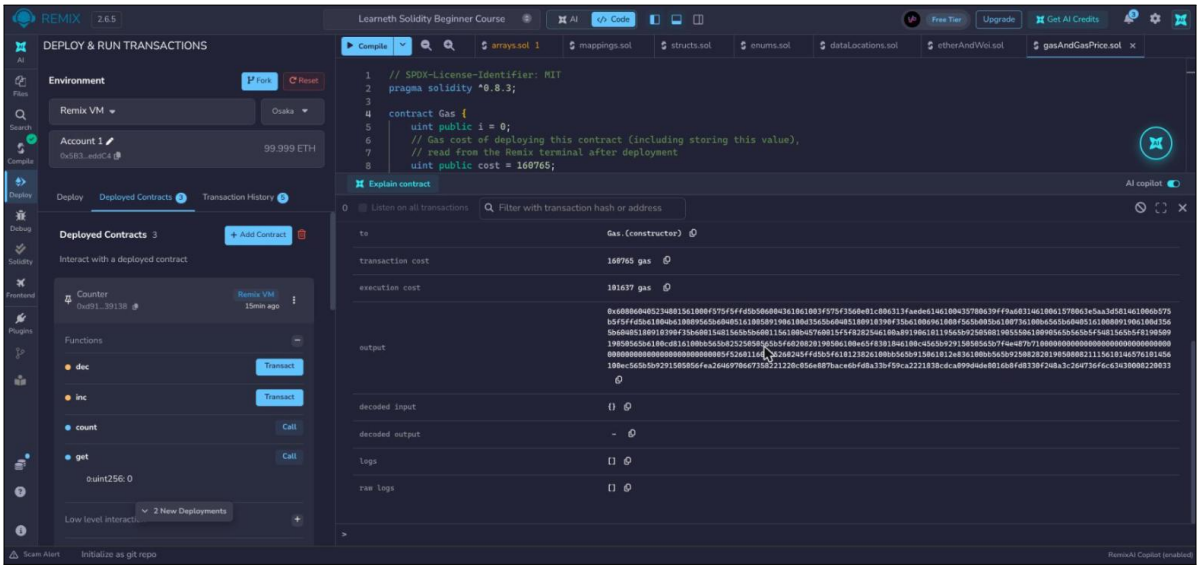


Figure 22: Gas contract deployment – transaction cost

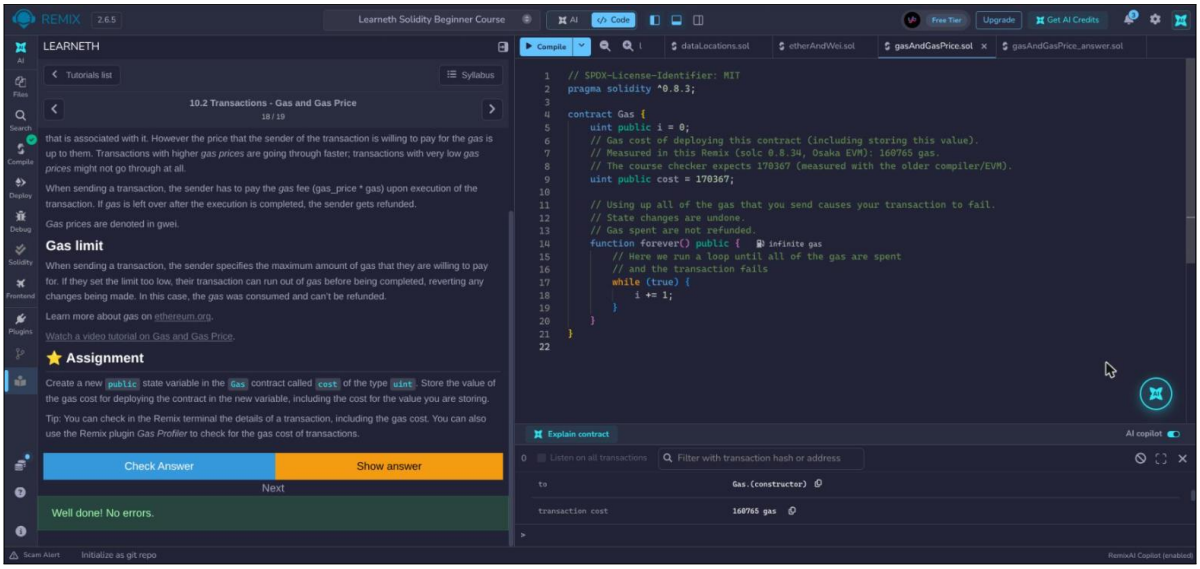


Figure 23: Gas and gas price

10.3 Sending Ether

```
contract Charity {
    address public owner;
    constructor() { owner = msg.sender; }
    function donate() public payable {}
    function withdraw() public {
        uint amount = address(this).balance;
        (bool sent, ) = owner.call{value: amount}("");
        require(sent, "Failed to send Ether");
    }
}
```

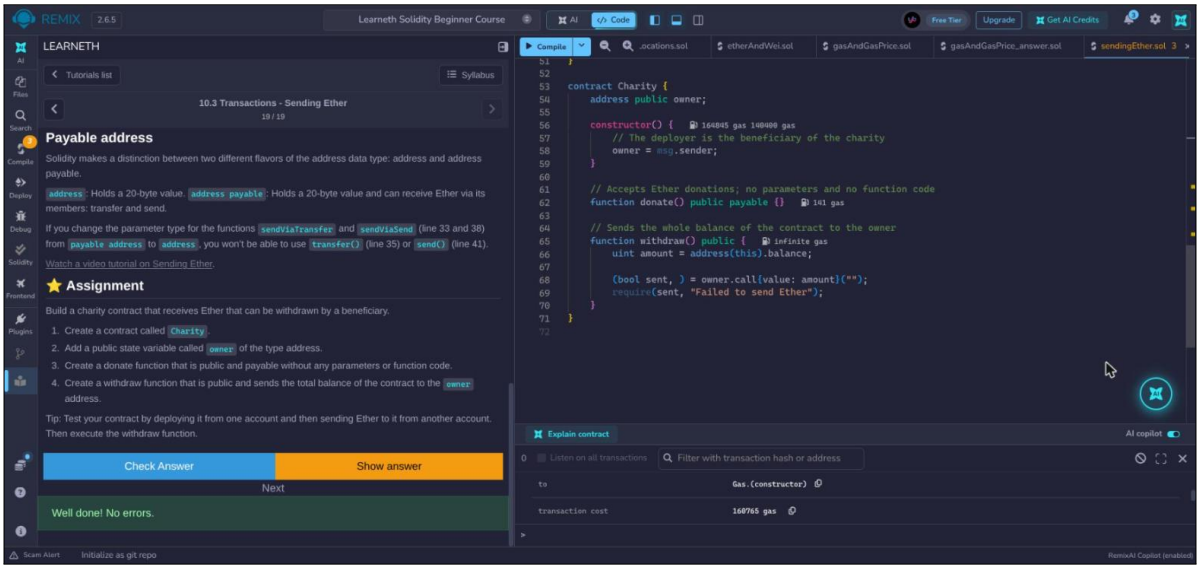


Figure 24: Sending Ether – Charity contract

Output

All 19 assignments were completed and accepted by the LearnEth checker.

Tutorial	Result
1 Compile, Deploy and Interact (Counter)	Contract compiled and deployed; `get()` returned 0, 1 after `inc()`, and 0 after `dec()`.
2 Basic Syntax	State variable `name` initialised to "Alice".
3 Primitive Data Types	`newAddr` holds an address, `neg` $= -12$, `newU` $= 0$ (uint8).
4 Variables	`blockNumber` stores the current `block.number`.
5.1 Reading and Writing	`get_b()` returns `true`.
5.2 View and Pure	State variable `x` is updated by `y`.
5.3 Modifiers and Constructors	The modifier performs the increment before the (empty) function body runs.
5.4 Inputs and Outputs	`returnTwo()` returns -2 and `true`.
6 Visibility	Child contract accesses the internal and public variables of Base.
7.1 If/Else	Returns `true` for even numbers, `false` for odd.
7.2 Loops	Loop runs 9 times; `count` $= 9$.
8.1 Arrays	`getArr()` returns [0, 1, 2].
8.2 Mappings	`set()` stores the account balance against the address.
8.3 Structs	Selected struct element in `todos` is reset to default values.
8.4 Enums	`getSize()` returns the current enum value.
9 Data Locations	`f()` returns (4, 1, 3): storage and memory copies are independent.
10.1 Ether and Wei	`oneGwei` $= 10^9$ wei; `isOneGwei` $=$ `true`.
10.2 Gas and Gas Price	Gas contract deployed; Remix showed a transaction cost of 160765 gas.
10.3 Sending Ether	Charity accepts donations via `donate()` and the owner withdraws the balance.

Note: In Tutorial 10.2 Remix measured 160765 gas for the deployment, but the course checker accepted only 170367 for `cost`, so that value was entered. The cause of the difference was not investigated.

Conclusion

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts.

Experiment completed successfully.